

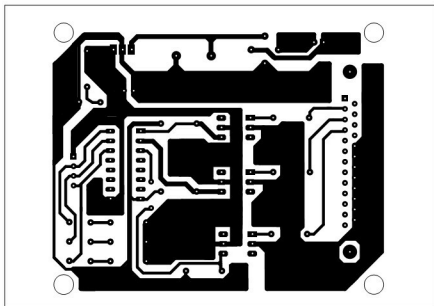


Fig. 5: A single-side, actual-size PCB layout for stepper motor controller using 'C'

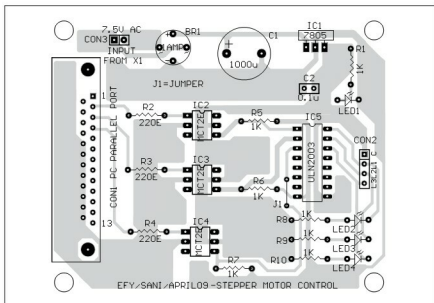


Fig. 6: Component layout for the PCB

3 of IC5. Output pins 15 and 14 of IC5 are connected to L2 and L3 coils of the stepper motor, respectively.

Three LEDs (LED2 through LED4) connected across the output of IC5 indicate switching of the stepper motor.

The circuit runs off 5V DC with maximum current rating of 500 mA derived from a common voltage regulator 7805.

Fig. 4 shows the pin details of the KT42JM4 stepper motor used in

the prototype.

Software program

The software (smc_v1.0.c) for controlling the stepper motor is written in 'C' language and compiled using Turbo C++ IDE. It requires the egavga.bgi and trip.chr graphic files for proper running. The program is DOS-based and runs under Windows 98. If you are using Windows XP/NT, for security reasons it cannot directly access

the port in DOS. In order to access the port, you have to use the userport.sys file, which can be downloaded from the Internet free of cost. Alternatively, you can use a Visual Basic program (smc_vb) along with the input32.dll file, which have been included in this month's EFY-CD for Windows NT/XP. The same has been implemented in Visual Basic6 for Windows NT/XP in the 'Multiple Device Switching Through Parallel Port' project published in May 2008 issue of EFY.

When the SMC_V1.0.exe program is run, it prompts you to enter the number of full rotations, direction and speed of running the motor.

User-defined functions cw(), acw() and speedf() are used for controlling the direction and speed of the stepper motor. The instruction outputb(0x0378, value) sends the data in binary format to the data port. When digit '1' (in decimal) is entered from the keyboard, it sends '001' (binary equivalent) to the stepper motor through the parallel port, and the motor moves one step. Similarly, when '2' is entered, it sends '010' to the motor and the motor moves the second step. Finally, when '4' is applied to the port, it sends '100' to the motor and the motor moves the third step. This completes one full sequence for running the motor through three steps for a 4-lead stepper motor. When this sequence (001 010 100) is further applied, the motor runs further in the same direction.

The motor runs through an angle of 7.5 degrees/step, so it requires $360/7.5=48$ steps for one complete rotation. By repeating these steps, it can be run through any desired number of rotations. This is done by using two 'for' loops in the program. The outer loop contains the number of rotations entered by the user and the inner loop repeats the steps for one complete rotation. Depending upon the outer loop, the inner loop repeats itself. Now, in order to run the motor in reverse direction, the sequence of the pulses has to be reversed, i.e., 2, 1, 4 in binary 010